

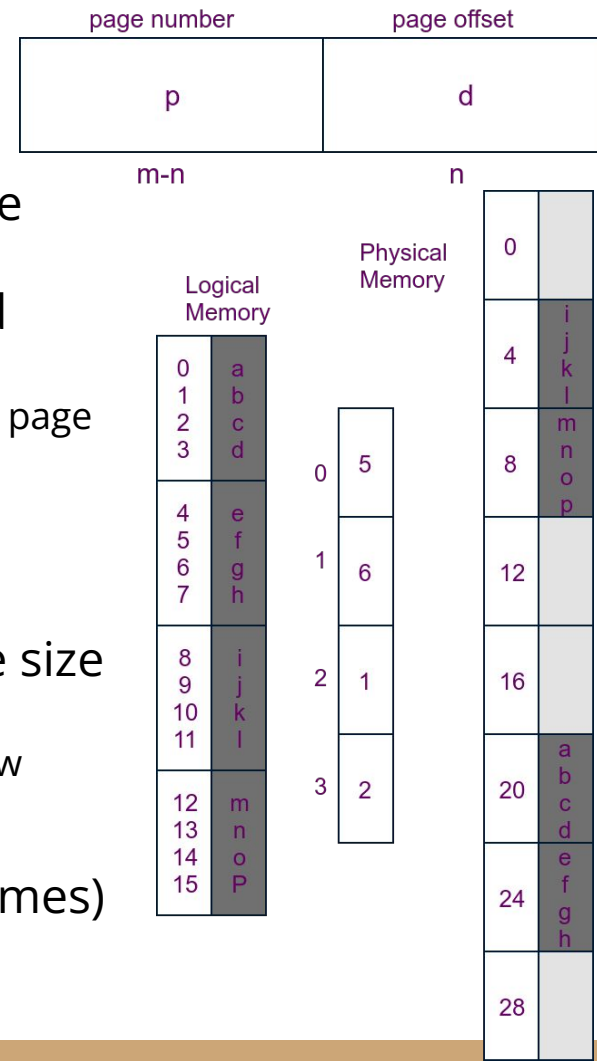
Paging

Dr. Jit Mukherjee



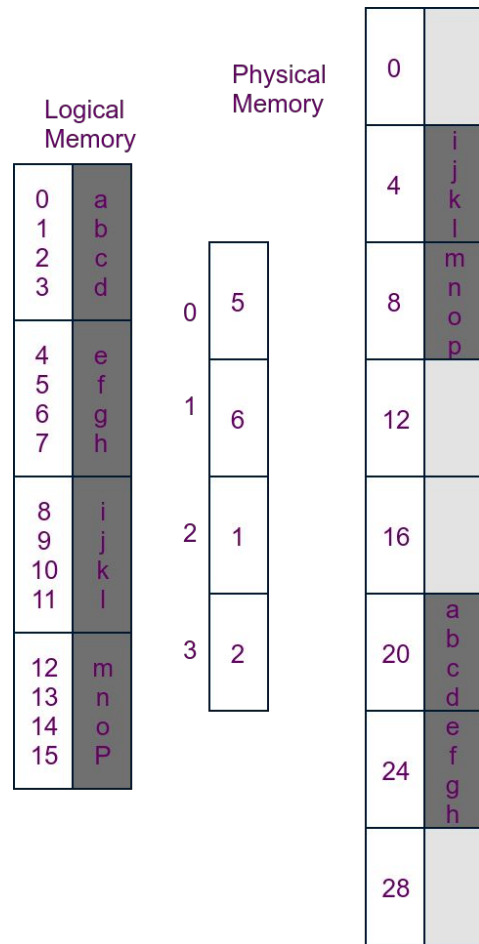
Basic Method

- Fixed sized blocks physical memory - Frames. Same sized logical memory - Pages
- Address generated by CPU -> Page number (p) and Page Offset (d)
 - Page number - Index of Page Table -> Base address of each page in physical memory
- Page size is defined by the hardware -> typically a power of 2.
 - 512 bytes - 16 MB
- If the size of logical address space is 2^m and a page size is 2^n bytes or words
 - High order $m-n$ bits of logical address -> page number. n low order bits -> page offset
 - p -> index to page table, d -> displacement
- Page size - 4 byte, Physical Memory - 32 byte (8 frames)
 - Logical address 0 -> Page 0, offset 0 -> Frame 5
 - Physical address = $5 \times 4 + 0 = 20$



Basic Method Contd.

- Logical address 3 -> offset 3 -> Physical = $5 \times 4 + 3$
- Logical address 10 -> page 2 offset 2 -> frame 1
 - Physical address = $1 \times 4 + 2 = 6$
- Paging very similar to a table of base registers
- No external fragmentation
 - Why?
- May be internal fragmentation
 - Because of the last frame
 - A process -> 77,766 bytes, Page size -> 2,048 bytes => 35 pages and 1,086 bytes
 - Internal fragmentation -> $2048 - 1086 = 962$ bytes
 - Worst case N page and 1 byte
- Consideration - internal fragmentation average $\frac{1}{2}$ page per process



Basic Method Contd.

- Is small page/frame size desirable? -> Overhead
 - Involved in each page table entry
 - Overhead reduces if size of page reduces
 - Disk I/O more efficient for large pages
- Generally Page size have grown over time. Ex - 4KB
- A process arrives in a system -> its size, expressed in pages examined -> Each page needs a frame -> for n pages, n frames. -> if available, it is allocated
- User view the memory as a single space. Mapping is hidden.
- OS takes care of the allocation of physical memory
 - Frame Table - one entry for each physical frame
- OS maintain a copy of the page table for each process
 - Dispatcher uses it
 - Translate logical address to physical whenever OS must map manually
- Paging increase context switching time

Hardware Support

- Most allocate a page table for each process. A pointer to the page table in PCB
 - Dispatcher reload user registers and define correct hardware page table value from user page table
- Simplest way -> Page table -> Dedicated Registers
 - Every access to memory must go through the paging map
 - Good if page table is small. Not feasible for large page table
- Page table is kept in main memory. *Page-table Base Register* -> page table
 - Changing page table needs changing this register -> reduce context switching
 - To access location i , must find index into page table -> PTBR offset -> further actual address
 - Two memory access is required -> slowed down by a factor of 2.
- Standard solution -> A special, small fast-lookup cache -> *Translation look-aside buffer (TLB)*

Hardware Support

- Each entry in TLB -> a key and a value
 - When the associated memory is presented with an item. The item is compared with all keys
 - If item found -> value is returned. Search -> Fast, Hardware -> Expensive
 - Typically entries in TLB -> [64, 1024]
- TLB contains a few page table item. Direct mapping of physical memory
- If page number is not there -> *TLB Miss*. A memory reference is made.
- If TLB full -> Replacement by LRU (least recently used).
 - *Wired Down* - can not be removed. Kernel call
- Address-space identifiers (ASID) -> Uniquely identify a process and provides address space protection for that process.
 - While searching virtual page number, ASID comes. If not is ASID -> TLB Miss
- 80% hit ratio - find desired page number 80% times
 - 20 nanosec to search in TLB. 100 nanosec to search in memory. Not found 20 nanosec + 100 nanosec to access page table + 100 nanosec to access byte
 - *Effective memory access time* - $0.80 \times 120 + 0.20 \times 220 = 140 \text{ nanoseconds}$ (40% slow down)

Protection

- One bit can define a page to read-write or read only.
- Protection bits check no write is performed in read only
- A valid-invalid bit
- Valid within address space.
Invalid not within
- May waste valuable space
- Page-table length register (PTLR) -> size of page table
 - Check logical address space

page 0
page 1
page 2
page 3
page 4
page 5

frame
number

0	V
1	I
Valid Invalid	

Page
Table

page 5	0
	1
page 0	2
	3
page 1	4
page 4	5
page 2	6
page 3	7
.	8
.	9
page n	